

Overview:

My language is called Lu. While my language currently utilizes the same basic reserved keywords as lox, I plan to expand some of these and make more stylistic choices in the upcoming labs as more lexical grammar emerges. I didn't yet implement some of these changes, such as additional keywords into the scanner level because I want to have their purposes clearly defined first in my lexical grammar. I did however, implement a newline token, multiline nested comments, multiline strings, and scientific notation into my language at the scanner level. So far, my design is inspired by some of the nuances of Python's syntax for readability and writability, but some of the rigidity of Lox for implementation.

Extra Credit Request:

For my language, I also implemented several features that go beyond a basic implementation. For instance, I have utilized a counter to incorporate nested C-style block comments. My language also accepts scientific notation for numbers, as shown in the Regex. My error handling also features a column and line number, and returns specific characters when relevant. Finally, I have incorporated escape characters into strings.

Lexical Grammar:

Regex for Identifiers: `[A-Za-z_][A-Za-z0-9_]*`

My language only allows identifiers to begin with alpha characters (including underscores), but then any subsequent characters can be alphanumeric.

Regex for String Literals (Excluding escape characters): `"[^"]*"'`

Regex for String Literals (Including escape characters): `"([^\"]|\\n|\\t|\\\\\\\\|\\\\\\")*"`

My language allows string literals to contain ordinary characters and supports four escape sequences: `\n`, `\t`, `\\`, and `\'`. These represent newlines, tabs, backslashes, and quotation marks, respectively.

Regex for Numbers: `(\d+(\.\d+)?)|([eE][+-]?\d+(\.\d+)?)`

My language allows for floating point numbers, integers, and scientific notation, although they are all stored under the NUMBER token type. All decimal numbers must have more digits following the decimal point so as not to get confused with the dot token. Leading decimals are also allowed in my language, and scientific notation can include an optional + or - character.

Setup Instructions:

The current setup of my language includes an endpoint in the primary file which counts the number of arguments and determines whether to enter REPL mode or execute a file. To setup my language Lu, you begin by opening the repository for the project, and then you can call:

```
python .\src\lu.py
```

to enter the REPL mode. In order to run a file, you must use the format:

```
python .\src\lu.py .\test\lab1\01_tokens.lu
```

(Being sure to replace the filename with the proper test 01-08c) This runs the test files I created.

Limitations:

All implemented test cases produced the expected output. Error cases also produced the expected diagnostics, including line and column information. Errors are handled using print statements, and REPL mode is still functional after receiving an error input. The primary limitation of my scanner is that I don't have other parts of the language implemented yet, so features such as lexical grammar will likely need to be extended later on as more rules are introduced.

Testing:

For all of the testing, the test cases are provided as Lu input in the repository under .\test\lab1\ There are 10 test case files, each containing a different aspect of the lexical grammar. Specific descriptions of the test cases are provided below, and all expected outputs match the actual outputs which are shown below.

Test 01: Punctuation and Operators. This test verifies that the scanner correctly recognizes all single-character punctuation tokens and multi-character operators, including !=, ==, <=, and >=.

```
TokenType.NEWLINE
None 1
TokenType.NEWLINE
None 2
TokenType.LEFT_PAREN ( None 3
TokenType.RIGHT_PAREN ) None 3
TokenType.LEFT_BRACE { None 3
TokenType.RIGHT_BRACE } None 3
TokenType.COMMA , None 3
TokenType.DOT . None 3
TokenType.MINUS - None 3
TokenType.PLUS + None 3
TokenType.SEMICOLON ; None 3
TokenType.STAR * None 3
TokenType.SLASH / None 3
TokenType.NEWLINE
None 3
TokenType.NEWLINE
None 4
TokenType.BANG ! None 5
TokenType.BANG_EQUAL != None 5
TokenType.NEWLINE
None 5
TokenType.EQUAL = None 6
TokenType.EQUAL_EQUAL == None 6
```

TokenType.NEWLINE
None 6
TokenType.LESS < None 7
TokenType.LESS_EQUAL <= None 7
TokenType.NEWLINE
None 7
TokenType.GREATER > None 8
TokenType.GREATER_EQUAL >= None 8
TokenType.EOF None 8

Test 02: Keywords and Identifiers. This test verifies that all reserved keywords are recognized correctly and that valid identifiers are distinguished from keywords, including identifiers that begin with or contain keyword text.

TokenType.NEWLINE
None 1
TokenType.NEWLINE
None 2
TokenType.NEWLINE
None 3
TokenType.AND and None 4
TokenType.CLASS class None 4
TokenType.ELSE else None 4
TokenType.FALSE false None 4
TokenType.FOR for None 4
TokenType.FUN fun None 4
TokenType.IF if None 4
TokenType.NIL nil None 4
TokenType.OR or None 4
TokenType.PRINT print None 4
TokenType.RETURN return None 4
TokenType.SUPER super None 4
TokenType.THIS this None 4
TokenType.TRUE true None 4
TokenType.VAR var None 4
TokenType.WHILE while None 4
TokenType.NEWLINE
None 4
TokenType.NEWLINE
None 5
TokenType.NEWLINE
None 6
TokenType.IDENTIFIER hello None 7
TokenType.NEWLINE
None 7
TokenType.IDENTIFIER _hello None 8
TokenType.NEWLINE
None 8
TokenType.IDENTIFIER hello123 None 9

```
TokenType.NEWLINE
None 9
TokenType.IDENTIFIER my_variable None 10
TokenType.NEWLINE
None 10
TokenType.IDENTIFIER _123 None 11
TokenType.NEWLINE
None 11
TokenType.NEWLINE
None 12
TokenType.NEWLINE
None 13
TokenType.IDENTIFIER variable None 14
TokenType.IDENTIFIER orchid None 14
TokenType.IDENTIFIER while_loop None 14
TokenType.IDENTIFIER andrew None 14
TokenType.IDENTIFIER classroom None 14
TokenType.IDENTIFIER elsewhere None 14
TokenType.IDENTIFIER falsehood None 14
TokenType.NEWLINE
None 14
TokenType.IDENTIFIER forest None 15
TokenType.IDENTIFIER function None 15
TokenType.IDENTIFIER iffy None 15
TokenType.IDENTIFIER nil_value None 15
TokenType.IDENTIFIER original None 15
TokenType.IDENTIFIER printer None 15
TokenType.IDENTIFIER return_value None 15
TokenType.IDENTIFIER superman None 15
TokenType.NEWLINE
None 15
TokenType.IDENTIFIER this_value None 16
TokenType.IDENTIFIER true_value None 16
TokenType.IDENTIFIER var_name None 16
TokenType.EOF None 16
```

Test 03: Number Literals. This test verifies that the scanner correctly recognizes integer, decimal, leading-dot, and scientific-notation number literals. It also tests boundaries between numeric literals and neighboring tokens.

```
TokenType.NEWLINE
None 1
TokenType.NEWLINE
None 2
TokenType.NEWLINE
None 3
TokenType.NUMBER 0 0.0 4
TokenType.NEWLINE
None 4
```

TokenType.NUMBER 1 1.0 5
TokenType.NEWLINE
None 5
TokenType.NUMBER 123 123.0 6
TokenType.NEWLINE
None 6
TokenType.NUMBER 999 999.0 7
TokenType.NEWLINE
None 7
TokenType.NUMBER 1000000 1000000.0 8
TokenType.NEWLINE
None 8
TokenType.NEWLINE
None 9
TokenType.NEWLINE
None 10
TokenType.NUMBER 0.0 0.0 11
TokenType.NEWLINE
None 11
TokenType.NUMBER 0.5 0.5 12
TokenType.NEWLINE
None 12
TokenType.NUMBER 1.0 1.0 13
TokenType.NEWLINE
None 13
TokenType.NUMBER 3.14 3.14 14
TokenType.NEWLINE
None 14
TokenType.NUMBER 123.456 123.456 15
TokenType.NEWLINE
None 15
TokenType.NUMBER 999.999 999.999 16
TokenType.NEWLINE
None 16
TokenType.NEWLINE
None 17
TokenType.NEWLINE
None 18
TokenType.NUMBER .1 0.1 19
TokenType.NEWLINE
None 19
TokenType.NUMBER .5 0.5 20
TokenType.NEWLINE
None 20
TokenType.NUMBER .123 0.123 21
TokenType.NEWLINE
None 21
TokenType.NUMBER .999 0.999 22
TokenType.NEWLINE

None 22
TokenType.NEWLINE
None 23
TokenType.NEWLINE
None 24
TokenType.NUMBER 1e5 100000.0 25
TokenType.NEWLINE
None 25
TokenType.NUMBER 1E5 100000.0 26
TokenType.NEWLINE
None 26
TokenType.NUMBER 1.5e5 150000.0 27
TokenType.NEWLINE
None 27
TokenType.NUMBER 1.5E5 150000.0 28
TokenType.NEWLINE
None 28
TokenType.NUMBER 1e+5 100000.0 29
TokenType.NEWLINE
None 29
TokenType.NUMBER 1e-5 1e-05 30
TokenType.NEWLINE
None 30
TokenType.NUMBER 1.5e+10 15000000000.0 31
TokenType.NEWLINE
None 31
TokenType.NUMBER 1.5e-10 1.5e-10 32
TokenType.NEWLINE
None 32
TokenType.NEWLINE
None 33
TokenType.NEWLINE
None 34
TokenType.NUMBER 123 123.0 35
TokenType.DOT . None 35
TokenType.IDENTIFIER foo None 35
TokenType.NEWLINE
None 35
TokenType.NUMBER 123 123.0 36
TokenType.DOT . None 36
TokenType.IDENTIFIER abc None 36
TokenType.NEWLINE
None 36
TokenType.NUMBER .123 0.123 37
TokenType.NEWLINE
None 37
TokenType.NUMBER 1.23 1.23 38
TokenType.NUMBER .45 0.45 38
TokenType.EOF None 38

Test 04: String Literals. This test verifies that the scanner correctly recognizes empty and non-empty strings, strings containing spaces and punctuation, multiline strings, and all supported escape sequences.

TokenType.NEWLINE

None 1

TokenType.NEWLINE

None 2

TokenType.NEWLINE

None 3

TokenType.STRING "hello" hello 4

TokenType.NEWLINE

None 4

TokenType.STRING "hello world" hello world 5

TokenType.NEWLINE

None 5

TokenType.STRING "" 6

TokenType.NEWLINE

None 6

TokenType.STRING "123" 123 7

TokenType.NEWLINE

None 7

TokenType.STRING "special characters: ! @ # \$ % ^ & * ()" special characters: ! @ # \$ % ^ & * () 8

TokenType.NEWLINE

None 8

TokenType.NEWLINE

None 9

TokenType.NEWLINE

None 10

TokenType.STRING "hello, world." hello, world. 11

TokenType.NEWLINE

None 11

TokenType.STRING "if (x >= 100)" if (x >= 100) 12

TokenType.NEWLINE

None 12

TokenType.NEWLINE

None 13

TokenType.NEWLINE

None 14

TokenType.STRING "line one

line two

line three" line one

line two

line three 15

TokenType.NEWLINE

None 17

TokenType.NEWLINE

None 18

TokenType.NEWLINE
None 19
TokenType.STRING "newline:\nsecond line" newline:
second line 20
TokenType.NEWLINE
None 20
TokenType.STRING "tab:\tindented" tab: indented 21
TokenType.NEWLINE
None 21
TokenType.STRING "quote: \"hello\"" quote: "hello" 22
TokenType.NEWLINE
None 22
TokenType.STRING "backslash: \" backslash: \" 23
TokenType.EOF None 23

Test 05: Comments and Whitespace. This test verifies that whitespace and comments are ignored by the scanner without affecting the tokens surrounding them. It includes line comments, block comments, multiline comments, and comments appearing between tokens.

TokenType.NEWLINE
None 1
TokenType.NEWLINE
None 2
TokenType.NEWLINE
None 3
TokenType.VAR var None 4
TokenType.IDENTIFIER x None 4
TokenType.EQUAL = None 4
TokenType.NUMBER 10 10.0 4
TokenType.NEWLINE
None 4
TokenType.VAR var None 5
TokenType.IDENTIFIER y None 5
TokenType.EQUAL = None 5
TokenType.NUMBER 20 20.0 5
TokenType.NEWLINE
None 5
TokenType.NEWLINE
None 6
TokenType.NEWLINE
None 7
TokenType.VAR var None 8
TokenType.IDENTIFIER a None 8
TokenType.EQUAL = None 8
TokenType.NUMBER 1 1.0 8
TokenType.NEWLINE
None 8
TokenType.VAR var None 9
TokenType.IDENTIFIER b None 9

TokenType.EQUAL = None 9
TokenType.NUMBER 2 2.0 9
TokenType.NEWLINE
None 9
TokenType.NEWLINE
None 10
TokenType.VAR var None 11
TokenType.IDENTIFIER c None 11
TokenType.EQUAL = None 11
TokenType.NUMBER 3 3.0 11
TokenType.NEWLINE
None 11
TokenType.NEWLINE
None 12
TokenType.NEWLINE
None 13
TokenType.NEWLINE
None 14
TokenType.NEWLINE
None 15
TokenType.NEWLINE
None 16
TokenType.NEWLINE
None 17
TokenType.VAR var None 18
TokenType.IDENTIFIER d None 18
TokenType.EQUAL = None 18
TokenType.NUMBER 4 4.0 18
TokenType.NEWLINE
None 18
TokenType.NEWLINE
None 19
TokenType.NEWLINE
None 22
TokenType.NEWLINE
None 23
TokenType.VAR var None 24
TokenType.IDENTIFIER e None 24
TokenType.EQUAL = None 24
TokenType.NUMBER 5 5.0 24
TokenType.NEWLINE
None 24
TokenType.NEWLINE
None 25
TokenType.NEWLINE
None 26
TokenType.VAR var None 27
TokenType.IDENTIFIER f None 27
TokenType.EQUAL = None 27

TokenType.NUMBER 6 6.0 27
TokenType.NEWLINE
None 27
TokenType.VAR var None 28
TokenType.IDENTIFIER result None 28
TokenType.EQUAL = None 28
TokenType.NUMBER 1 1.0 28
TokenType.PLUS + None 28
TokenType.NUMBER 2 2.0 28
TokenType.NEWLINE
None 28
TokenType.NEWLINE
None 29
TokenType.NEWLINE
None 30
TokenType.VAR var None 31
TokenType.IDENTIFIER g None 31
TokenType.EQUAL = None 31
TokenType.NUMBER 7 7.0 31
TokenType.PLUS + None 31
TokenType.NUMBER 8 8.0 31
TokenType.NEWLINE
None 31
TokenType.VAR var None 32
TokenType.IDENTIFIER division None 32
TokenType.EQUAL = None 32
TokenType.NUMBER 10 10.0 32
TokenType.SLASH / None 32
TokenType.NUMBER 2 2.0 32
TokenType.NEWLINE
None 32
TokenType.NEWLINE
None 33
TokenType.EOF None 34

Test 06: Nested Block Comments. This test verifies that block comments can contain nested block comments and that the scanner correctly tracks the nesting depth before continuing to scan the remaining source code.

TokenType.NEWLINE
None 1
TokenType.NEWLINE
None 2
TokenType.NEWLINE
None 6
TokenType.NEWLINE
None 7
TokenType.NEWLINE
None 8

TokenType.NEWLINE
None 19
TokenType.NEWLINE
None 20
TokenType.EOF None 21

Test 07: Integration. This test combines keywords, identifiers, literals, operators, punctuation, comments, whitespace, and expressions in a single source file to verify that the scanner correctly handles multiple lexical features together.

TokenType.NEWLINE
None 1
TokenType.NEWLINE
None 2
TokenType.VAR var None 3
TokenType.IDENTIFIER name None 3
TokenType.EQUAL = None 3
TokenType.STRING "Lucy" Lucy 3
TokenType.NEWLINE
None 3
TokenType.VAR var None 4
TokenType.IDENTIFIER age None 4
TokenType.EQUAL = None 4
TokenType.NUMBER 20 20.0 4
TokenType.NEWLINE
None 4
TokenType.NEWLINE
None 5
TokenType.NEWLINE
None 6
TokenType.VAR var None 7
TokenType.IDENTIFIER total None 7
TokenType.EQUAL = None 7
TokenType.NUMBER 10 10.0 7
TokenType.PLUS + None 7
TokenType.NUMBER 20 20.0 7
TokenType.STAR * None 7
TokenType.NUMBER 3 3.0 7
TokenType.NEWLINE
None 7
TokenType.VAR var None 8
TokenType.IDENTIFIER average None 8
TokenType.EQUAL = None 8
TokenType.IDENTIFIER total None 8
TokenType.SLASH / None 8
TokenType.NUMBER 2 2.0 8
TokenType.NEWLINE
None 8

```

TokenType.VAR var None 9
TokenType.IDENTIFIER passed None 9
TokenType.EQUAL = None 9
TokenType.TRUE true None 9
TokenType.NEWLINE
None 9
TokenType.NEWLINE
None 10
TokenType.IF if None 11
TokenType.LEFT_PAREN ( None 11
TokenType.IDENTIFIER passed None 11
TokenType.AND and None 11
TokenType.BANG ! None 11
TokenType.IDENTIFIER eligible None 11
TokenType.RIGHT_PAREN ) None 11
TokenType.LEFT_BRACE { None 11
TokenType.NEWLINE
None 11
TokenType.PRINT print None 12
TokenType.STRING "Ready" Ready 12
TokenType.NEWLINE
None 12
TokenType.RIGHT_BRACE } None 13
TokenType.NEWLINE
None 13
TokenType.NEWLINE
None 14
TokenType.VAR var None 15
TokenType.IDENTIFIER message None 15
TokenType.EQUAL = None 15
TokenType.STRING "Hello\nworld" Hello
world 15
TokenType.EOF None 15

```

Test 08a: Unexpected Characters. This test verifies that the scanner detects and reports characters that are not valid components of the language's lexical grammar, including the corresponding line and column information.

```

[line 3, column 1] Error: Unexpected character. '@'
[line 3, column 3] Error: Unexpected character. '#'
[line 3, column 5] Error: Unexpected character. '$'
[line 3, column 7] Error: Unexpected character. '%'
[line 3, column 9] Error: Unexpected character. '&'
[line 3, column 11] Error: Unexpected character. '?'
[line 3, column 13] Error: Unexpected character. '~'
[line 3, column 15] Error: Unexpected character. "'"
TokenType.NEWLINE
None 1

```

TokenType.NEWLINE
None 2
TokenType.EOF None 3

Test 08b: Unterminated Strings. This test verifies that the scanner detects strings that reach the end of the source without a closing quotation mark, including multiline strings and strings containing escape sequences.

[line 8, column 1] Error: Unterminated string.

TokenType.NEWLINE
None 1
TokenType.NEWLINE
None 2
TokenType.NEWLINE
None 3
TokenType.EOF None 8

Test 08c: Unterminated Block Comments. This test verifies that the scanner detects block comments that reach the end of the source without a closing delimiter, including multiline and nested block comments.

[line 6, column 21] Error: Unterminated block comment.

TokenType.NEWLINE
None 1
TokenType.NEWLINE
None 2
TokenType.EOF None 6